Module - 9:
Consistency & Advanced Patterns

- Amjad Hossain
12th April, 2026

Strong Consistency

Definition:

After a successful write -

- **all subsequent reads must return that latest value**
- no stale reads allowed

Mental Model

Once it’s done, it’s visible everywhere immediately.

Strong Consistency

Example — Bank Account Transfer (Single DB)

- User transfers **$100**
- System:
  - Debit sender
  - Credit receiver
- Both operations are wrapped in a **single ACID transaction**

Guarantees	**Implementation Characteristics**
● No partial state ● No stale reads ● Always correct	● Synchronous ● Often uses: ○ Transactions (ACID) ○ Locks ○ 2PC (2 phase commit in distributed DBs)

Strong Consistency (Trade-offs)

Benefit	Cost
Always correct	High latency

Strong Consistency (Trade-offs)

Benefit	Cost
Always correct	High latency
Simple	Low scalability

Strong Consistency (Trade-offs)

Benefit	Cost
Always correct	High latency
Simple	Low scalability
No anomalies	Tight coupling

Eventual Consistency

Definition:

System may -

- **return stale data temporarily**
- will converge to correct state **eventually**

Mental Model

It will be correct… just not immediately.

Eventual Consistency

Example - Bank Transfer (Distributed Services)

Services:

- Account Service
- Ledger Service
- Notification Service

Flow	**Temporary State**
● Debit event published ● Credit happens asynchronously ● Read models update later	● Sender debited ● Receiver not credited yet

Eventual Consistency (Trade-offs)

Benefit	Cost
Scalable	Temporary inconsistency

Eventual Consistency (Trade-offs)

Benefit	Cost
Scalable	Temporary inconsistency
Decoupled	Complex debugging

Eventual Consistency (Trade-offs)

Benefit	Cost
Scalable	Temporary inconsistency
Decoupled	Complex debugging
Resilient	Requires retry/idempotency

A Problem… …

Scenario: Global Card System Across Regions

Imagine:

- User has $100
- Two transactions happen almost simultaneously:
  - 🇧🇩 Dhaka POS : $100
  - 🇺🇸 New York POS : $100

A Problem… …

If You Enforce Strong Consistency Globally

To prevent double-spend every transaction must:

- Lock the same global account
- Across regions

Latency Explosion	**Availability Issues**
• Cross-region round trip • Payment takes too long (unacceptable)	• If central system is down → **no payments work globally**

A Problem… …

If You Enforce Strong Consistency Globally

To prevent double-spend every transaction must:

- Lock the same global account
- Across regions

Latency Explosion	**Availability Issues**
• Cross-region round trip • Payment takes too long (unacceptable)	• If central system is down → **no payments work globally**

CAP Theorem !!

Hybrid model

Strong Consistency (Local / Fast)	Eventual Consistency (Global Reconciliation)
● Maintain: ○ Available balance locally ● Use: ○ Atomic reservation (hold amount) This gives **instant approval/decline**	● Transactions propagated asynchronously ● Systems reconcile: ○ Across regions ○ Across ledgers

Hybrid model (What Can Still Happen)

Rare edge cases:	Handled by:
● Temporary double authorization ● Out-of-sync balance across regions	● Reconciliation jobs ● Reversal transactions ● Fraud systems

Few helpful patterns

- **single-row pessimistic lock** on the balance/account record (ref: the link)
- **optimistic concurrency** with version checking and retry (ref: the link)
- **atomic compare-and-swap style update** in storage (ref: the link)

Messaging Patterns

1. Point-to-Point (Queue)
 	
● One producer → one consumer
● Used for:
  ○ Task processing
  ○ Jobs		

Messaging Patterns

1. Point-to-Point (Queue)	2. Publish–Subscribe	
● One producer → one consumer ● Used for: ○ Task processing ○ Jobs	● One producer → many consumers ● Used for: ○ Notifications ○ Event-driven systems	

Messaging Patterns

1. Point-to-Point (Queue)	2. Publish–Subscribe	3. Event Streaming
● One producer → one consumer ● Used for: ○ Task processing ○ Jobs	● One producer → many consumers ● Used for: ○ Notifications ○ Event-driven systems	● Events stored and replayed ● Consumers read independently ● Used for: ○ Event Sourcing

Message Brokers

RabbitMQ

- Good for:
  - Commands
  - Task queues
  - Request/response async

Message Brokers

Kafka

- Good for:
  - Event sourcing
  - Analytics
  - High throughput streams

Problem with CRUD (single model)

Same model handles:

- Writes (complex validation, invariants)
- Reads (fast queries, different shapes)

Problem with CRUD (single model)

Same model handles:

- Writes (complex validation, invariants)
- Reads (fast queries, different shapes)

Consequences:

- Bloated domain models
- Complex joins for reads
- Poor performance under scale

Problem with CRUD (single model)

Same model handles:

- Writes (complex validation, invariants)
- Reads (fast queries, different shapes)

Consequences:

- Bloated domain models
- Complex joins for reads
- Poor performance under scale

Observation:

Read and write concerns have different characteristics

Command Query Responsibility Segregation

Command side (Write)

- Changes state
- Enforces business rules
- Returns minimal response (often just success/failure)

Command Query Responsibility Segregation

Command side (Write)

- Changes state
- Enforces business rules
- Returns minimal response (often just success/failure)

Query side (Read)

- Returns data
- Optimized for UI/use-cases
- No business mutations

Command Query Responsibility Segregation

Command side (Write)

- Changes state
- Enforces business rules
- Returns minimal response (often just success/failure)

Query side (Read)

- Returns data
- Optimized for UI/use-cases
- No business mutations

Separate models, *possibly separate data stores*

CQRS (Read Model Design)

Goal: Optimize for queries, not normalization

Examples:

- Flattened tables
- Pre-joined views
- Cached projections

Why:

- Avoid complex joins at runtime
- Faster API responses

CQRS Flow (simple)

- Client sends **Command**
- Domain validates (invariants)
- Persist to **Write DB**
- Update **Read Model** (sync or async)
- Client queries via **Query API**

Read model may be **denormalized**

CQRS + Messaging (Common Setup)

After command succeeds:

- Publish **event** (e.g., `OrderCreated`)

Read side subscribes:

- Updates projections(denormalized view data)

Enables:

- Loose coupling
- Async scaling

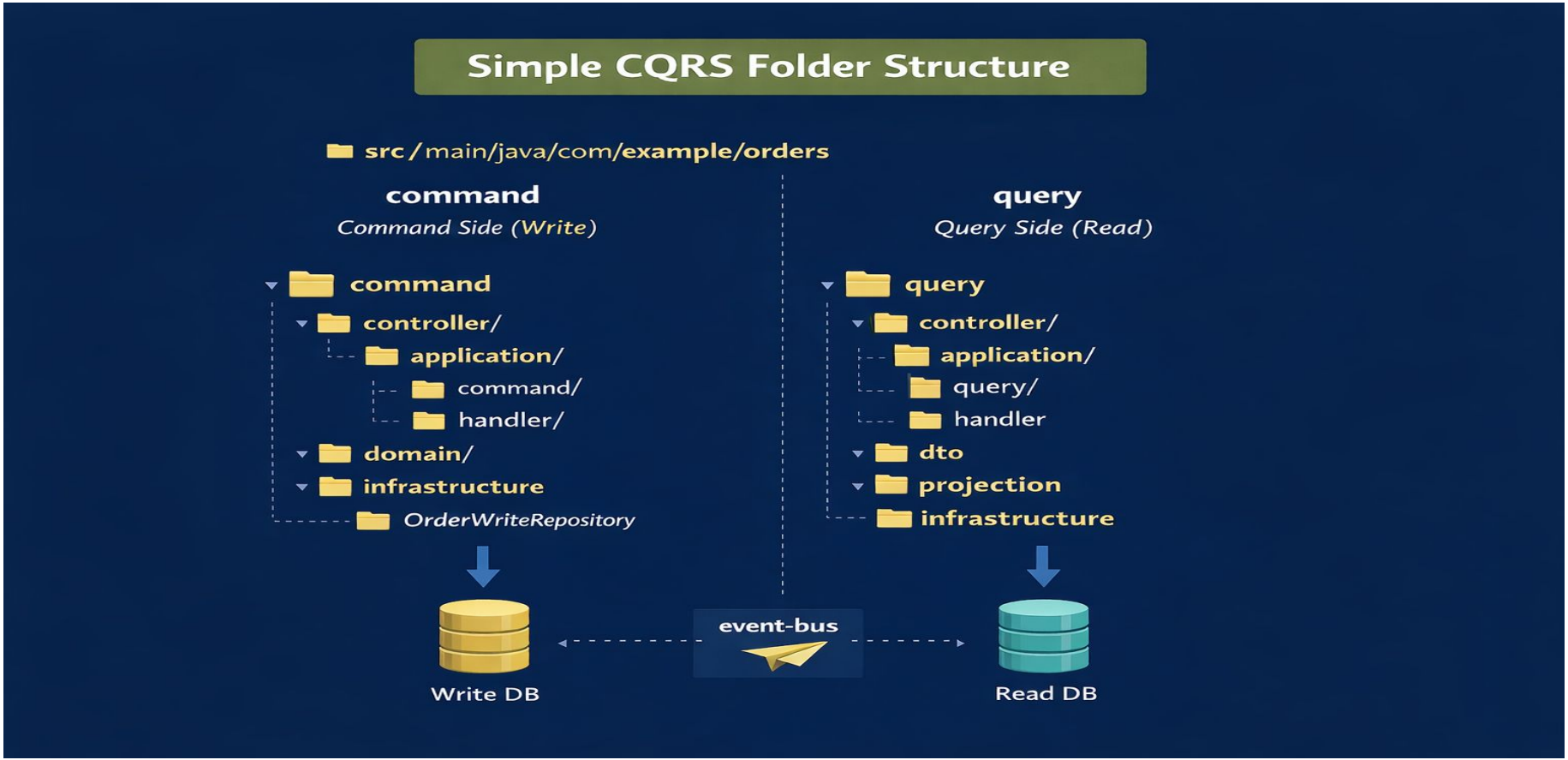
CQRS + Event Sourcing (Advanced)

- Command → produces **events**
- Events stored in **Event Store**
- Read model built from events

Write model = **events**

Read model = **projections**

Command Query Responsibility Segregation



Transaction

A transaction means:

- a group of operations
- treated as one logical unit
- either all succeed
- or all fail

In a single database, we usually rely on **ACID**

Example

Transfer money from Account A to Account B:

- debit A
- credit B

If debit succeeds but credit fails, the system is wrong.
So the database transaction ensures both happen together.

Distributed Transactions

In microservices:

- Order Service has its own DB
- Payment Service has its own DB
- Inventory Service has its own DB

Now a single business action may span multiple services.

Example

Place Order:

- Order Service creates order
- Payment Service charges payment
- Inventory Service reserves stock
- Shipping Service prepares shipment

This is now a **distributed transaction problem**.

Why This is Hard

Because there is:

- no single database transaction
- no shared commit boundary
- no single lock manager
- network failures between services
- independent service availability

So a business flow may fail in the middle.

Example

- Order created
- Payment charged
- Inventory reservation failed

Traditional Solution: Two-Phase Commit (2PC)

Phase 1 - Prepare

Coordinator asks all participants: “Are you ready to commit?”

Each participant:

- performs local preparation
- locks resources
- says yes or no

Phase 2 - Commit / Rollback

If all say yes:	If any says no:
• coordinator tells all to commit	• coordinator tells all to rollback

Why 2PC is Problematic in Microservices

1. **Tight coupling**
   - All services must participate in the same transaction protocol.
2. **Blocking**
   - If coordinator is slow or fails, participants may stay blocked.
3. **Poor scalability**
   - Locks are held longer, reducing throughput.
4. **Availability impact**
   - One slow/down service can stall the whole business flow.
5. **Different services may use -**
   - different databases
   - different brokers
   - different storage models

Saga Pattern

A **Saga** is a sequence of **local transactions** across services.

Each step:

- updates its own local database
- publishes an event or sends a command

If a later step fails:

- previous successful steps are not rolled back automatically by DB
- instead, the system runs **compensating actions**

So Saga is:

> Do step-by-step, and if something fails later, undo previous business effects logically.

Saga Example: Order Placement

Let’s take a simple flow:

1. **Order Service** creates order
2. **Payment Service** charges customer
3. I**nventory Service** reserves items
4. **Shipping Service** creates shipment

Success path

- Order created
- Payment completed
- Stock reserved
- Shipment created

Saga Example: Order Placement

Failure path

Suppose shipping fails.

Then compensation may be:

- cancel inventory reservation
- refund payment
- cancel order

Saga Types

There are two main styles:

1. **Choreography**
2. **Orchestration**

Saga Types: Choreography

- Services react to each other’s events.
- No central controller.

Example

- Order Service publishes `OrderCreated`
- Payment Service listens and charges payment
- Payment Service publishes `PaymentCompleted`
- Inventory Service listens and reserves stock

Each service knows when to react.

Saga Types: Choreography

Pros

- loose coupling in control flow
- no single central coordinator
- naturally event-driven

Saga Types: Choreography

Pros

- loose coupling in control flow
- no single central coordinator
- naturally event-driven

Cons

- business flow becomes hard to visualize
- debugging is more difficult
- logic gets scattered across services
- can create event-chain complexity

Saga Types: Choreography

Pros

- loose coupling in control flow
- no single central coordinator
- naturally event-driven

Cons

- business flow becomes hard to visualize
- debugging is more difficult
- logic gets scattered across services
- can create event-chain complexity

Choreography is elegant at first, but can become invisible complexity.

Saga Types: Orchestration

A central orchestrator controls the flow.

Example

- Saga Orchestrator sends command to Payment Service
- then sends command to Inventory Service
- then sends command to Shipping Service
- if failure occurs, orchestrator triggers compensations

Saga Types: Orchestration

Pros

- flow is explicit
- easier to understand
- easier to monitor and debug
- better for complex business process

Saga Types: Orchestration

Pros

- flow is explicit
- easier to understand
- easier to monitor and debug
- better for complex business process

Cons

- orchestrator becomes central dependency
- slightly more control coupling
- one more component to maintain

Saga Types: Orchestration

Pros

- flow is explicit
- easier to understand
- easier to monitor and debug
- better for complex business process

Cons

- orchestrator becomes central dependency
- slightly more control coupling
- one more component to maintain

Orchestration trades some decentralization for clarity.

Important Limitation of Saga

Saga does **not** give instant global consistency.

Instead it gives:

- local correctness per step
- eventual business consistency after all steps or compensations finish

This means temporary intermediate states can exist.

Example

For a short period:

- payment captured
- order still pending shipment

That is normal in Saga-based architecture.

Common Problems in Saga

1. Duplicate message delivery

A compensation or step may run more than once.

Solution

- idempotent handlers

Common Problems in Saga

2. Out-of-order events

Events may not arrive in the expected order.

Solution

- versioning
- state machine checks

Common Problems in Saga

3. Partial compensation failure

Refund may fail after inventory release succeeded.

Solution

- retries
- dead-letter handling
- manual recovery workflows

Common Problems in Saga

4. Long-running transactions

Saga may take seconds, minutes, or even longer.

Solution

- explicit status tracking
- timeout handling

Good Saga Design

A proper Saga usually needs:

- clear business states
- idempotent commands/events
- compensating actions defined upfront
- observability and tracing
- retry strategy
- timeout strategy
- dead-letter queue or recovery mechanism

Saga: State Machine Thinking

Saga is easier to design if you think in states.

Example Order states

- PENDING
- PAYMENT_COMPLETED
- INVENTORY_RESERVED
- SHIPMENT_CREATED
- CANCELLED
- REFUNDED

Each event moves the workflow from one valid state to another.

This makes the design easier to reason about.

Saga: Where Fits Best

Use Saga when:

- one business process spans multiple services
- each service owns its own database
- compensation is possible

Good examples

- order processing
- booking systems
- payment workflows
- shipment workflows
- loan application flow

Saga: is a Bad Fit for

Avoid Saga when:

- everything is still in one database
- simple local transaction is enough
- compensation is impossible or legally unsafe
- workflow is too trivial to justify distributed complexity

Do not use Saga just because you use microservices.

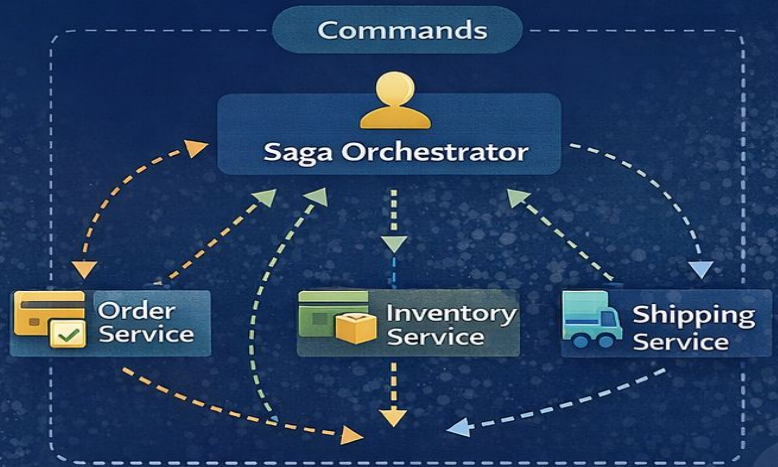
Saga Pattern: Choreography vs Orchestration

Choreography

Event-Based Coordination

- Decentralized control
- Services react to events
- Can be hard to manage as it scales

Orchestration



Command-Based Coordination

- Centralized control
- Orchestrator directs flow
- Easier to understand and monitor

Another Problem

Traditional approach:

- Store only **current state**

Example:

Account Balance = 500

Problem:

- No history
- Hard to debug
- Weak auditability
- Cannot reconstruct past state

Core Idea of Event Sourcing

Store **what happened**, not just **what is**

Instead of:

Balance = 500

We store:

+1000 (Deposit)
-500 (Withdrawal)

Current state = **replayed result of events**

What is an Event?

An **event** is:

- Something that **already happened**
- Immutable
- Named in **past tense**

Examples:

- `PaymentProcessed`
- `OrderCreated`
- `InventoryReserved`

Event Store

It’s -

- Append-only storage
- Stores sequence of events

Properties:

- Immutable (no update/delete)
- Ordered
- Source of truth

Think:

- Database = “current state”
- Event Store = “history of truth”

How State is Built

State is not stored directly. It is **reconstructed**:

1. Load all events
2. Apply them one by one
3. Build current state

Example Flow

Event 1: AccountCreated (balance = 0)

Event 2: Deposited 1000

Event 3: Withdrawn 500

Final Balance = 500

Event Sourcing Flow

1. Command received
2. Validate using current state
3. Produce event
4. Store event
5. Update projections (read model)

State = derived, not primary

Projections (Read Models)

Projections are:

- Derived views from events
- Optimized for queries

Examples:

- Account summary
- Order list
- Dashboard data

Snapshotting

To avoid replaying all events:

- Store periodic snapshots

Example:

- Snapshot at event 100 → balance = 800
- Then replay from 101 onward

Event Sourcing: Key Benefits

1. Full Audit Trail

- Every change is recorded

Event Sourcing: Key Benefits

1. Full Audit Trail

- Every change is recorded

2. Time Travel

- Rebuild state at any point in time

Event Sourcing: Key Benefits

1. Full Audit Trail

- Every change is recorded

2. Time Travel

- Rebuild state at any point in time

3. Debugging Power

- Replay events to reproduce bugs

Event Sourcing: Key Benefits

1. Full Audit Trail

- Every change is recorded

2. Time Travel

- Rebuild state at any point in time

3. Debugging Power

- Replay events to reproduce bugs

4. Flexibility

- Create new projections later

Event Sourcing: Trade-offs

Benefit	Cost
Full history	Storage growth
Replay capability	Complexity
Flexible reads	Event versioning issues
Strong audit	Harder queries

Event Sourcing: When NOT to Use

Avoid when:

- Simple CRUD system
- No audit requirement
- Small application
- Team not experienced

Overkill in many cases

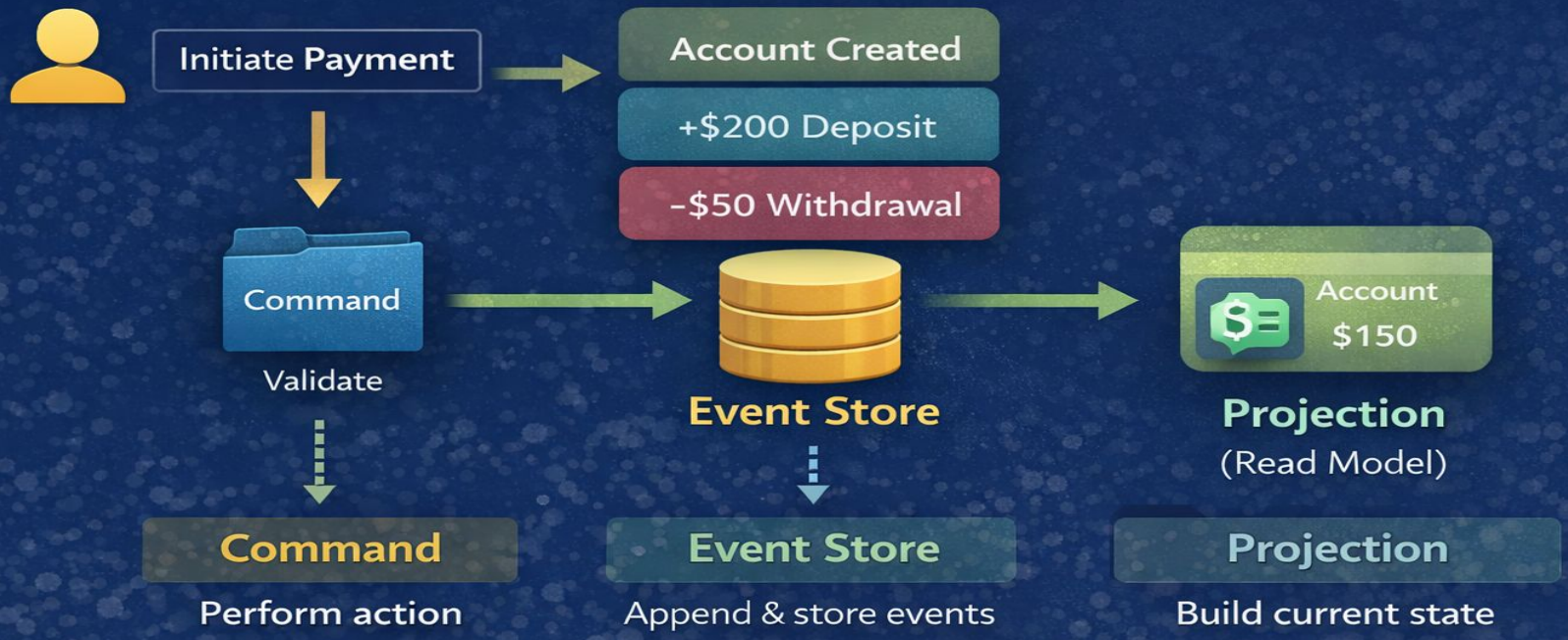
Event Sourcing: When to Use

Use when:

- Strong audit requirement
- Financial systems
- Complex business workflows
- Need for replay/debugging
- Event-driven architecture

Event Sourcing

Capture Changes as Immutable Events



Thank you